

# Миграционен план — инфраструктура на МАБИ

## Контекст

External developer-ът приключва на **1 август 2026**. Към момента на плана разполагаме с **~2 месеца**. Платформата е **live** с **~350 активни клиента**, а **уебинар се провежда всеки месец** — downtime в уебинарен ден е недопустим.

Мигрират се **3 сървъра** към нова инфраструктура (доставчикът е по наш избор).

| Сървър (текущ)        | Съдържа                | Важни детайли                                                    |
|-----------------------|------------------------|------------------------------------------------------------------|
| cPanel (Superhosting) | tonkin.bg — WordPress  | Managed, <b>без root</b> , MySQL, ~15GB                          |
| Contabo VPS #1        | my.tonkin.bg — Next.js | Node.js, env файлове с API ключове, ~2GB                         |
| Contabo VPS #2        | Video Streamer         | 100GB+ HLS, PHP + ffmpeg + cron, Zoom webhook incoming.tonkin.bg |

## Водещи принципи

- 1 Минимален / нулев downtime** — особено около месечния уебинар (нулев толеранс в уебинарен ден).
- 2 Обратимост** — всяка стъпка има rollback; старият сървър остава жив, докато новият не е потвърден.
- 3 Резерв преди 1 август** — приключваме с резервно време, докато external dev-ът е още наличен за предаване на знанието и достъпи.
- 4 Тествай преди DNS превключване** — никога „на съляпо“.
- 5 Консолидация на администрирането** — където е разумно, един доставчик/подход, за да е поддръжката проста за екипа, който поема.

## 1. Целева инфраструктура (за всеки сървър + защо)

Целевият доставчик не е фиксиран. Водещата идея е **правилният инструмент за всеки сървър**, а не единен доставчик за всичко: където managed услуга върши по-добра работа — ползва се тя; където е нужен self-managed сървър — VPS. Изборът зависи и от профила на екипа, който поема (технически vs не).

### 1.1 tonkin.bg (WordPress): два варианта според профила на екипа

WordPress не изисква непременно root. Изборът зависи от това дали екипът, който поема, има техническа компетентност.

#### Вариант А — Managed WordPress (cPanel хост / Cloudways)

- Какво: WP на cPanel хост или на Cloudways (managed WordPress върху cloud VPS). Управление на плъгини, бекъпи, SSL, email — наготово.
- Плюс: ниско натоварване за поддръжка, WordPress-оптимизирана среда.
- Минус: по-малко контрол, без root, премия в цената при premium хостове.

#### Вариант Б — Self-hosted (root VPS + Docker)

- Какво: root VPS (SuperHosting / Hetzner / др.), WordPress + MariaDB в Docker, зад Nginx Proxy Manager.
- Плюс: пълен контрол, инфраструктура като код, консолидация с останалите услуги, по-ниска цена.
- Минус: екипът поема цялата поддръжка — OS, сигурност, бекъпи, кеширане, ъпдейти.

**Препоръка спрямо екипа:**

- **Изцяло нетехнически екип** → **Вариант А** (cPanel / managed). Без DevOps капацитет self-hosting на WordPress е риск, не предимство.
- **Екип с техническа / DevOps компетентност** → **Вариант Б** (контрол + консолидация + цена).

## 1.2 my.tonkin.bg (Next.js): Contabo VPS → Vercel

**Препоръка:** Vercel (Pro).

**Защо:**

- Next.js е **роден на Vercel** — нулева поддръжка, автоматичен build/deploy от Git, **preview deployment**-и за всеки PR.
- **Вградено криптирано управление на env променливи** (per environment) — по-сигурно от env файлове на VPS (виж т.4).
- Глобален CDN, авто-скейл. Малкият размер (~2GB) и stateless природата го правят идеален кандидат.

**Алтернатива:** Hetzner + Docker (ако екипът държи „всичко на едно място“). Възможно е, но за Next.js Vercel печели по простота и сигурност на ключовете.

**Зависимост:** четете от WP REST API → при миграция на WP трябва да обновим API URL-а (env). Затова редът има значение (т.2).

**Кеширане:** четенето от WP REST API се кешира (ISR / fetch cache в Next.js) — разтоварва WordPress, ускорява клиентската зона и намалява Vercel function invocations (и разхода).

## 1.3 Video Streamer: Contabo VPS → Hetzner VPS + CDN (storage по два варианта)

Видеообработката (ffmpeg, Whisper, cron, Zoom webhook) изисква постоянен self-managed сървър. Доставка на 100GB+ HLS минава през **CDN**. Провайдер — **Hetzner**: видеото генерира много изходящ трафик, а Hetzner го включва евтино.

**Storage — два варианта за самите файлове:**

**Вариант А (препоръчан старт) — HLS на VPS volume + CDN.** Файловете остават на сървъра (допълнителен диск), CDN ги кешира отпред. По-малко движещи се части — по-нисък риск за екип, който поема непознат код.

**Вариант Б (по-чист дългосрочно) — HLS на обектно хранилище (R2) + CDN.** Файловете се изнасят в Cloudflare R2 (S3 API, нулев egress, CDN-friendly); сървърът само ги обработва. По-евтино при мащаб и прави бъдещи миграции тривиални, но изисква промяна как се пишат/сервират файловете.

*Zoom webhook (incoming.tonkin.bg) остава на VPS-а. Storage Box е само за студен архив, не за стрийминг (склад ≠ streaming origin).*

**Защо този избор:** обработката няма serverless алтернатива → self-managed VPS; egress-ът на видеото налага CDN + евтин трафик (Hetzner); започваме с по-простия Вариант А, а Б е естествената еволюция.

## 2. Ред на миграция (кой пръв — най-безопасно)

**Принцип:** редът върви от **най-нискорисковия и най-обратим** компонент към **най-критичния (данните)** — новата инфраструктура се валидира с по-безопасните миграции, преди да се стигне до необратимите стъпки.

**Препоръчан ред:**

### 1 Next.js (my.tonkin.bg) — пръв.

Малък, (почти) stateless, лесно обратим (DNS/CNAME). Vercel preview позволява пълно тестване без риск за продукцията. Продължава да четете от **стария** WordPress, така че миграцията му не засяга останалите

услуги. Подходящ като пилотна миграция за валидиране на новата инфраструктура.

## 2 Video Streamer — втори.

Изолиран, но тежък (100GB) и сложен (cron, webhook). Първоначалният rsync се изпълнява **фоново в продължение на дни**; същинският cutover — в **не-уебинарен** прозорец. Изолираността позволява **паралелен пробег** (стар + нов) преди финално превключване.

## 3 WordPress — последен.

Най-висок обхват на щетата: 15GB данни, 350 потребители, **източник на истината** за Next.js. Правим го последен — когато новата инфраструктура е вече валидирана, а zero-downtime процедурата — утвърдена. След него обновяваме API URL-а на Next.js (вече на Vercel) към новия WP.

*Защо не WP пръв: макар всичко да зависи от него, рискът му е най-голям. „Safest first“ означава да започнем с ниско-рисковото, не с гръбнака.*

**Около уебинарите:** фиксираме месечната уебинарна дата; **никакъв cutover ±1 ден** около нея; Video cutover-ът задължително далеч от уебинар.

## 3. WordPress — миграция с нулев downtime + тест преди DNS

Стъпките важат и за двата варианта от т.1.1 (managed или self-hosted).

**Принципът на zero downtime:** старият сайт обслужва трафика през цялото време; новият се изгражда и тества паралелно; реалното превключване е само смяна на DNS (бързо заради ниския TTL) плюс финален delta sync — така прозорецът на застъпване е сведен до минимум.

- 1 **Намаляване на DNS TTL** на записа за tonkin.bg до 300s няколко дни предварително — за да се разпространи cutover-ът за минути, а не часове.
- 2 **Изграждане** на целевата среда (managed план или root VPS + Docker WordPress + MariaDB) — готова предварително, преди да се пипа продукцията.
- 3 **Пренос на съдържанието** — базата (~15GB) се изважда с mysqldump през SSH (не през phpMyAdmin в браузъра — твърде голяма), а wp-content (теми, плъгини, качени файлове) — с rsync (инкрементално, устойчиво на прекъсване). cPanel няма root → преносът е „pull“ (издърпване от стария към новия).
- 4 **Вдигане зад временен адрес** — staging поддомейн (staging.tonkin.bg) или локален hosts override, сочещ tonkin.bg към новия IP само за тествания.
- 5 **Изчерпателен тест** на временния адрес — ръчно по критичните потоци (вход, **Tonkin LMS** курс/урок, форми/плащания), **curl/Postman** на REST API пътищата, които Next.js консумира (сравнение стар vs нов отговор), и преглед на логовете за PHP грешки. WP-CLI search-replace за URL-ите при нужда.
- 6 **Финален delta sync** на базата непосредствено преди cutover (минимален прозорец); по избор кратък read-only режим на стария за консистентност.
- 7 **Превключване на DNS** към новия — единствената точка на реална смяна; бързо заради ниския TTL, с незабавен мониторинг след това.
- 8 **Запазване на стария cPanel** няколко дни — rollback при проблем, преди изваждането му от експлоатация.

**Тест преди DNS switch:** чрез hosts override / staging поддомейн се минават критичните потоци, без да се пипа живият трафик; потвърждава се, че REST API за Next.js работи идентично.

**Запазване на SEO (бонус — извън поисканото).** Миграцията е на същия домейн, така че при коректно изпълнение SEO позициите се запазват. Ключови мерки:

- Идентична URL структура; при промяна на URL → **301 redirect** (стар → нов).

- При cutover: премахване на noindex / robots.txt Disallow от новата среда (иначе Google деиндексира сайта); staging остава noindex срещу duplicate content.
- Запазване на sitemap.xml, canonical и meta таговете; същият HTTPS; производителност поне колкото преди (Core Web Vitals).
- След cutover: resubmit на sitemap в Google Search Console + следене на crawl грешки / 404.

---

## 4. Next.js (Vercel) — миграция + сигурно управление на ключове

- 1 Свързване на Git репото към **Vercel**.
- 2 Пренос на env променливите в **Vercel Environment Variables**, разделени per environment (Production / Preview / Development). Ключовете не влизат в репото.
- 3 Deploy и тест на **Vercel preview URL** (без да се пипа my.tonkin.bg); потвърждаване на четенето от WP REST API.
- 4 Закачане на домейна my.tonkin.bg (CNAME към Vercel) + превключване на DNS.
- 5 Старият Contabo VPS остава като rollback няколко дни.

### Сигурно управление на ключове:

- Ключовете живеят във **Vercel encrypted store** — не в env файлове на диск, не в Git.
- В репото влиза само .env.example (шаблон без стойности); реалните стойности са във Vercel / .env.local (gitignored).
- NEXT\_PUBLIC\_ **префикс** прави променливата видима в браузърния bundle — тайните ключове са **без** него (server-only: само в API routes / server компоненти).
- Тайните се ползват само server-side — заявките към WP API с ключове минават през Next.js сървър, не от браузъра.
- Разделяне per environment (preview ≠ production ключове); минимални права.
- **Ротация на всички ключове** по време на миграцията — старите env файлове са стояли на VPS (възможна експозиция).
- API URL-ът към WordPress е env променлива → обновява се след миграцията на WP (т.3).

---

## 5. Video Streamer — 100GB+ файлове + cron без прекъсване на стрийминга

Рискове: 100GB файлове, активни cron jobs, Zoom webhook (пропуснат webhook = загубен запис), стриймингът не бива да спира.

- 1 **Изграждане** на новия VPS: ffmpeg, deploy на PHP/Node скриптовете, репликиране на crontab — **изключен първоначално**, за да не дублира.
- 2 **Доставка през CDN/object storage** на HLS файловете — така преместването им е прозрачно за зрителите.
- 3 **Първоначален rsync** на 100GB, фоново в продължение на дни (в tmux / systemd, независимо от SSH сесията; старият сървър продължава да стриймва). rsync е инкрементален и **възобновяем** — прекъсване не води до загуба: повторното пускане продължава от спряното, с --partial за недовършените файлове, без презапис на вече прехвърленото.
- 4 **Финален инкрементален rsync (delta)** в **не-убинарен** прозорец.
- 5 **Пренасочване на Zoom webhook** (incoming.tonkin.bg) към новия сървър — тест с тестово Zoom събитие. Това е най-критичната стъпка.

- 6 По избор: **паралелен пробег** — старият продължава да обработва, новият е валидиран — преди пълно превключване.
- 7 Превключване на DNS за стрийминг/webhook поддомейна; **изключване на cron на стария** след cutover — за да не върви една и съща задача на двата сървъра; запазване като fallback.

**Защо стриймингът не спира:** доставка през CDN/object storage прави местенето на файлове прозрачно; rsync с жив стар сървър до cutover; cutover далеч от уебинар.

## 6. Топ-3 риска + митигации

### 1 Downtime / пропуснат запис в уебинарен ден.

Митигация: всички cutover-и далеч от уебинарните дати ( $\pm 1$  ден резерв); паралелен пробег на видео webhook-a; готов rollback.

### 2 Загуба / несъответствие на WordPress данни при миграцията (15GB, 350 потребители, възможни записи по време на cutover).

Митигация: delta sync + кратък maintenance прозорец; верифицирани бекъпи; нисък DNS TTL; rollback към cPanel.

### 3 Изтичане / неправилна ротация на API ключове при Next.js миграцията.

Митигация: ротация на всички ключове; Vercel encrypted store; никога в Git; минимални права.

*Допълнителен риск — предаване на знанието: external dev напуска 1 август; custom кодът (Tonkin LMS, темата, AI ментор, видео pipeline) може да е недокументиран. Всички достъпи и документация се вземат преди напускането; миграцията приключва с резерв.*

## 7. Timeline по седмици (юни → началото на август)

*Cutover-ите се планират извън уебинарните седмици.*

| Седмица | Период       | Дейности                                                                                                             |
|---------|--------------|----------------------------------------------------------------------------------------------------------------------|
| 1       | юни, седм. 1 | Достъпи/credentials от external dev; одит на 3-те сървъра; избор на целеви доставчици; фиксиране на уебинарните дати |
| 2       | юни, седм. 2 | Изграждане на новата инфраструктура; <b>Next.js</b> → <b>Vercel</b> (preview тест)                                   |
| 3       | юни, седм. 3 | <b>Next.js cutover</b> (DNS); мониторинг; стабилизация                                                               |
| 4       | юни, седм. 4 | Video: нов VPS, скриптове, cron; старт на фонов rsync (100GB)                                                        |
| 5       | юли, седм. 1 | Video: финален delta sync + webhook пренасочване (не-уебинарен прозорец); паралелен пробег                           |
| 6       | юли, седм. 2 | <b>Video cutover</b> + стабилизация                                                                                  |
| 7       | юли, седм. 3 | WordPress: целева среда + пробна миграция (DB + файлове) + staging тест                                              |
| 8       | юли, седм. 4 | <b>WordPress cutover</b> (нисък TTL, delta, DNS); обновяване на Next.js API URL                                      |

| Седмица | Период          | Дейности                                                                                                                    |
|---------|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
| 9       | август, седм. 1 | Резерв/стабилизация; изваждане на старите сървъри от експлоатация; финална документация — <b>преди external dev напуска</b> |